

William Van Uiter

CICS 160 OOP

Cole Reilly

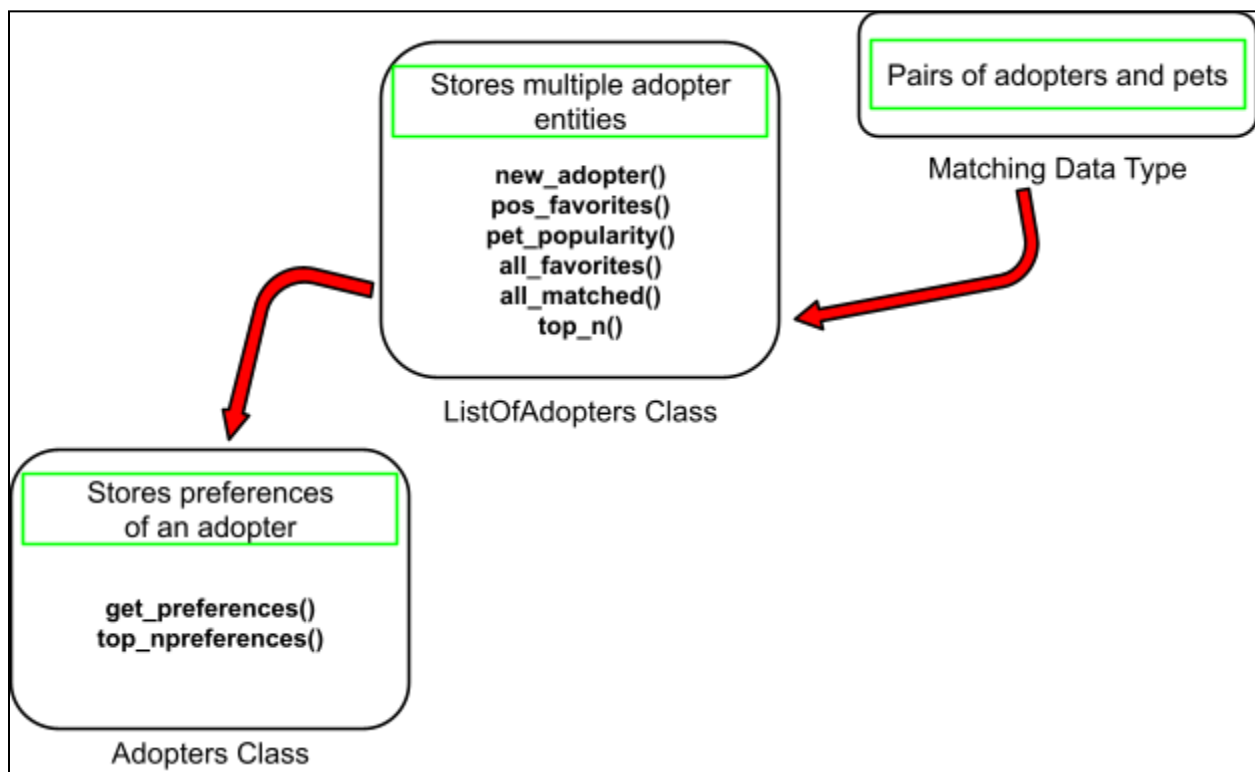
3 October 2026

Self-Check:

Defect	Where it could have appeared in my design	How I avoided it
D1	Adopter.get_preferences()	I made sure to describe the type of object that was being returned as well as what information the return object contained.
D2	Pet and adopter indexing vs. pet ranks	I stated at the top of ListOfAdopters that indexing is 0 based and ranking is 1 based.
D3	ListOfAdopters.all_matched()	I made sure to specify the function checks if every adopter in the internal self.adopters list is in matching (i.e. was very clear about how each variable was used)
D4	ListOfAdopters.new_adopter(), Adopter.get_preferences()	I made sure to state that new_adopter() appends (mutates) to the list and that get_preferences returns a new copy.
D5	Matching is built without duplicates and pairs 1 adopter with 1 pet.	I stated that matching was expected to be a proper object of type Matching.
D6	ListOfAdopters.top_n() sounds like it returns the top n pets.	I made sure to state that top_n checks if adopters are matched with a pet in their top n and the name still

		loosely pertains to what the function does.
D7	n index being out of range	I stated that when an index n is used the method will raise an IndexError if n is not a proper index for the task.
D8	ListOfAdopters.pet_popularity() could have described how the ranking was computed	pet_popularity() instead says what the return type for ranking looks like.

Diagram:



Design Justification:

- **Single Responsibility:** the Adopters class stores the preferences of a singular adopter entity. Additionally, the ListOfAdopters class stores preferences for multiple adopters and compares it to matching parameters. I chose to keep ListOfAdopters as one class because

the relationship between it and Matching is more a component of a composed-of relationship rather than a is-a relationship.

- Dependency Inversion: My design does not explicitly say how information is ultimately collected, only that it is passed in as a parameter. For example, my ADTs only say that Matching is passed in and what it is expected to look like, while leaving the construction of Matching to another class. Additionally, the documentation only specifies functionality of the code but not what specific actions are happening internally, leaving implementation more open for the programmer.